

ログイン画面でログインできない現象の修正

CORS（Cross-Origin Resource Sharing）によるエラーの解消

デベロッパーツールのコンソール上に、赤い文字で書かれている `has been blocked by CORS policy` というメッセージをご覧ください。

これが、ログイン画面でログインできない現象の最たる原因といえます。

このエラーは何を意味しているのか

これは「Aという場所（ポート5173）」にある画面から、「Bという場所（ポート8080）」にあるAPIを叩こうとしたときに、ブラウザが「勝手に他のサーバーのデータを操作させないように」と、安全のために通信を遮断している状態です。

GitHub Codespacesのような複雑なURL環境では、Spring Boot側で「このURL（フロントエンド）からのアクセスは信頼できるから通していいよ」と明示的に許可する必要があります。

解決策：Spring Boot側の設定修正

そのため、`SecurityConfig.java` にCORSを許可するための設定を組み込む必要があります。現在の`securityFilterChain` メソッドに `cors` の設定を追加してください。

```
// インポートが必要な場合は追加してください
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.Arrays;

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        // 1. CORS設定を有効化
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .csrf(csrf -> csrf.disable())
        .headers(headers -> headers.frameOptions(frame -> frame.disable()))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/h2-console/**", "/api/users/**", "/api/attendance/**").permitAll()
            .anyRequest().authenticated()
        );
}
```

```
        return http.build();
    }

    // 2. CORSの具体的な許可ルールを定義するメソッド
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        // フロントエンドのURLを許可（ワイルドカード "*" を使うか、正確なURLを指定します）
        // Codespaces環境では "*" を指定するのが最も確実です
        configuration.setAllowedOriginPatterns(Arrays.asList("*"));

        // 許可するメソッド（GET, POSTなど）
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "DELETE", "OPTIONS"));

        // 許可するヘッダー
        configuration.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type", "X-Requested-With"));

        // クッキーなどの認証情報を許可するか（今回はfalseでOK）
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration); // 全パスに適用
        return source;
    }
}
```

401 Unauthorized によるエラーの解消

この `401 Unauthorized` というエラーは、CORS（通信の門番）ではなく、Spring Security（認証の門番）が「君のことは知っている（リクエストは届いた）けれど、そのログイン情報は正しくないよ」と正式に拒否している状態です。

これには2つの可能性が考えられます。

パスワード照合の不一致（BCrypt）

現在、`UserAccountService.java` でパスワードをチェックする際、どのように記述していますか？

もし `curl` で登録した際に `BCryptPasswordEncoder` を使ってハッシュ化して保存している場合、ログイン時も `matches` メソッドを使って照合する必要があります。

【チェック】 `UserAccountService.java` のログインメソッド

以下のようなメソッドが `UserAccountService.java` に含まれているか確認してください。

```
public UserAccount login(String userId, String rawPassword) {  
    UserAccount user = repository.findById(userId)  
        .orElseThrow(() -> new RuntimeException("User not found"));  
  
    // rawPassword (入力された平文) と user.getPassword() (DBのハッシュ化済み) を比較  
    if (passwordEncoder.matches(rawPassword, user.getPassword())) {  
        return user;  
    } else {  
        throw new RuntimeException("Invalid password");  
    }  
}
```

注意点: 以前の `curl` コマンドでパスワードを登録した際、もし「ハッシュ化を通さずにDBに保存した」場合は、DB内は平文（`mysecretpassword`）になっているはずです。一方で、プログラムがハッシュ照合（`matches`）を試みると、形式が合わずに必ず失敗します。

Vue.jsから送る「名前」の不一致

Vue.jsの `LoginForm.vue` で送っているデータの「キー名」と、Javaのコントローラーで受け取っている「変数名」が一致していない可能性があります。

【チェック1】Vue.js側（ `LoginForm.vue` ）

```
const response = await apiClient.post('/users/login', {  
  userId: userId.value,    // ここが "userId"  
  password: password.value // ここが "password"  
});
```

【チェック2】 Java側 (`UserAccount` クラスなど)

```
@Data // GetterやSetterを自分で書く必要がなくなる
public class UserAccount {

    // ** 省略 **

    @Column(nullable = false, unique = true)
    private String userId; // メールアドレス[cite: 1]

    @Column(nullable = false)
    private String password; // BCryptハッシュ化済みパスワード[cite: 1]

    // ** 省略 **
}
```


真犯人を特定するためのデバッグ方法

Spring Boot側の `UserAccountController` のログインメソッド内に、以下のログを一行追加して再起動してみてください。

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    // ↓ これを追加して、Vueから何が届いているかターミナルで確認
    System.out.println("ログイン試行: ID=" + request.getUserId() + ", PW=" + request.getPassword());

    // ... 既存の処理
}
```

ログを確認して判断

- IDやPWが `null` になっている → 容疑者B（名前の不一致）です。
- IDやPWは正しく表示される → 容疑者A（パスワード照合ロジックのミス）です。